

tf.strided_slice Stay organized with collections Save and categorize content based on your preferences.

https://www.tensorflow.org/api_docs/python/tf/strided_slice

Extracts a strided slice of a tensor (generalized Python array indexing).

Function Signatures

```
tf.strided_slice( input_, begin, end, strides=None,
begin_mask=0, end_mask=0, ellipsis_mask=0, new_axis_mask=0,
shrink_axis_mask=0, var=None, name=None )
```

Code Examples

```
tf.strided_slice(
input_,
begin,
end,
strides=None,
begin_mask=0,
end_mask=0,
ellipsis_mask=0,
new_axis_mask=0,
shrink_axis_mask=0,
var=None,
name=None
)
```

```
tf.strided_slice(
input_,
begin,
end,
strides=None,
begin_mask=0,
end_mask=0,
ellipsis_mask=0,
new_axis_mask=0,
shrink_axis_mask=0,
var=None,
name=None
)
```

```
t = tf.constant([[[1, 1, 1], [2, 2, 2]],
[[3, 3, 3], [4, 4, 4]],
[[5, 5, 5], [6, 6, 6]]])
tf.strided_slice(t, [1, 0, 0], [2, 1, 3], [1, 1, 1]) # [[[3, 3,
3]]]
tf.strided_slice(t, [1, 0, 0], [2, 2, 3], [1, 1, 1]) # [[[3, 3,
3],
# [4, 4, 4]]]
tf.strided_slice(t, [1, -1, 0], [2, -3, 3], [1, -1, 1]) # [[[4, 4,
4],
# [3, 3, 3]]]
```

```
t = tf.constant([[[1, 1, 1], [2, 2, 2]],
[[3, 3, 3], [4, 4, 4]],
[[5, 5, 5], [6, 6, 6]]])
tf.strided_slice(t, [1, 0, 0], [2, 1, 3], [1, 1, 1]) # [[[3, 3,
3]]]
tf.strided_slice(t, [1, 0, 0], [2, 2, 3], [1, 1, 1]) # [[[3, 3,
3],
# [4, 4, 4]]]
tf.strided_slice(t, [1, -1, 0], [2, -3, 3], [1, -1, 1]) # [[[4, 4,
4],
# [3, 3, 3]]]
```

Documentation

Extracts a strided slice of a tensor (generalized Python array indexing).

View aliases

Compat aliases for migration

See

Migration guide for
more details.

`tf.compat.v1.strided_slice`

See also `tf.slice`.

Instead of calling this op directly most users will want to use the NumPy-style slicing syntax (e.g. `tensor[..., 3:4:-1, tf.newaxis, 3]`), which is supported via `tf.Tensor.getitem` and `tf.Variable.getitem`.

The interface of this op is a low-level encoding of the slicing syntax.

Roughly speaking, this op extracts a slice of size $(\text{end} - \text{begin}) / \text{stride}$ from the given input_ tensor. Starting at the location specified by `begin` the slice continues by adding `stride` to the index until all dimensions are not less than `end`.

Note that a stride can be negative, which causes a reverse slice.

Given a Python slice `input[spec0, spec1, ..., specn]`, this function will be called as follows.

`begin`, `end`, and `strides` will be vectors of length `n`.

`n` in general is not equal to the rank of the `input_` tensor.

In each mask field (`begin_mask`, `end_mask`, `ellipsis_mask`, `new_axis_mask`, `shrink_axis_mask`) the `i`th bit will correspond to the `i`th spec.

If the `i`th bit of `begin_mask` is set, `begin[i]` is ignored and the fullest possible range in that dimension is used instead. `end_mask` works analogously, except with the end range.

`foo[5:,:3]` on a 7x8x9 tensor is equivalent to `foo[5:7,0:8,0:3]`.

`foo[::-1]` reverses a tensor with shape 8.

If the `i`th bit of `ellipsis_mask` is set, as many unspecified dimensions as needed will be inserted between other dimensions. Only one non-zero bit is allowed in `ellipsis_mask`.

For example `foo[3:5,...,4:5]` on a shape 10x3x3x10 tensor is equivalent to `foo[3:5,:,:,4:5]` and `foo[3:5,...]` is equivalent to `foo[3:5,:,:,:]`.

If the `i`th bit of `new_axis_mask` is set, then `begin`, `end`, and `stride` are ignored and a new length 1 dimension is added at this point in the output tensor.

For example,
`foo[:4, tf.newaxis, :2]` would produce a shape (4, 1, 2) tensor.

If the `i`th bit of `shrink_axis_mask` is set, it implies that the `i`th specification shrinks the dimensionality by 1, taking on the value at index `begin[i]`. `end[i]` and `strides[i]` are ignored in this case. For example in

Python one might do `foo[:, 3, :]` which would result in `shrink_axis_mask` equal to 2.

Returns